ITCS383 Software Construction and Evolution

Project Phase 2 Part 2 (The Maintenance) Description

Score: 10% of the total course score
Timeline: 2-30 April 2026
Due: 30 April 2026 at 23.55 (commit time on GitHub)
Presentation: 1 May 2026 9:00 - 12:00, in-class (make-up class)

Context

After you have successfully handed over the project, each group will continue maintaining it. Changes will be requested for your project, and you will need to update the software to accommodate them.

Group Pairs

Product Owner	Project Maintainer
JITNONGNOOONG	Aaaaaaa
Aaaaaaa	FolkliyGrunt
FolkliyGrunt	Chongyai
Chongyai	JianCha
JianCha	Arai Kor Dai
Arai Kor Dai	Emerald
Emerald	Bughair
Bughair	Motrakor
Motrakor	MharRuengSang
MharRuengSang	JITNONGNOOONG

Change Requests

Each group must work on the following 3 additional features.

1. **Feature 1 – Mobile Client App:** Each group must add a new client platform, a mobile application platform focusing solely on the native Android platform, to their project. The mobile application must include all the functionalities of the original web application (including all the user-facing functionalities). The backend part can be the existing one, with some changes to support this new client platform. You can create a new, separate repository specifically for this mobile application. Please include the link to the new repository in the README.md of the main repository.

2. **Features 2 and 3 – Two New Features:** Each group must ask its product owner (see the table of group pairs above) for 2 new changes (i.e., features) they want to

add to the product. These two features must be implemented in both the web and mobile applications.

Note: When you take on the product owner role, please ensure you request **reasonable features that are feasible within the given time frame to your friend's group**. You will also be graded on the features that you request.

Deliverables

- **D1: The Working Prototype with Full CI Pipeline**
  - You must demonstrate that you successfully complete each change request by having a workable prototype of the system (both web and mobile versions).
  - The backend must be successfully deployed on the cloud (choose a free-tier service like Render or Vercel).
  - There must be GitHub Actions that run automated builds and tests. It must also include SonarQube Cloud, which checks code quality whenever a new push is made.

- **D2: The Quality Check**
  - You must demonstrate that your changes to the project do not introduce any issues or degrade its code quality. Create a file called D2_CODE_QUALITY.md to show this.
  - You must run SonarQube after the changes and report its results in D2_CODE_QUALITY.md, comparing them with the version before the changes (you can take snapshots of the SonarQube report before and after).
  - Make sure you properly set up the new code in SonarQube to analyze only the "new code" (see more information on how to set up this here https://docs.sonarsource.com/sonarqube-server/10.6/project-administration/clean-as-you-code-settings/defining-new-code). You may need to separate the project in SonarQube due to differences in programming languages and infrastructure. If that is the case, include the analysis of all the repositories you have.
  - The test coverage of the new code must be more than 90%.

- **D3: Change Request Analysis**
  - You must analyze the features that you must complete in this phase. Then, break them into change requests. Each feature may contain multiple change requests, depending on the type of feature request.
  - You must follow the change request schema introduced in the lecture "Evolution and Maintenance Models" (slide 36) and create a CR table for each change request, similar to the one shown below. Put them in the file called D3_CHANGE_REQUESTS.md.

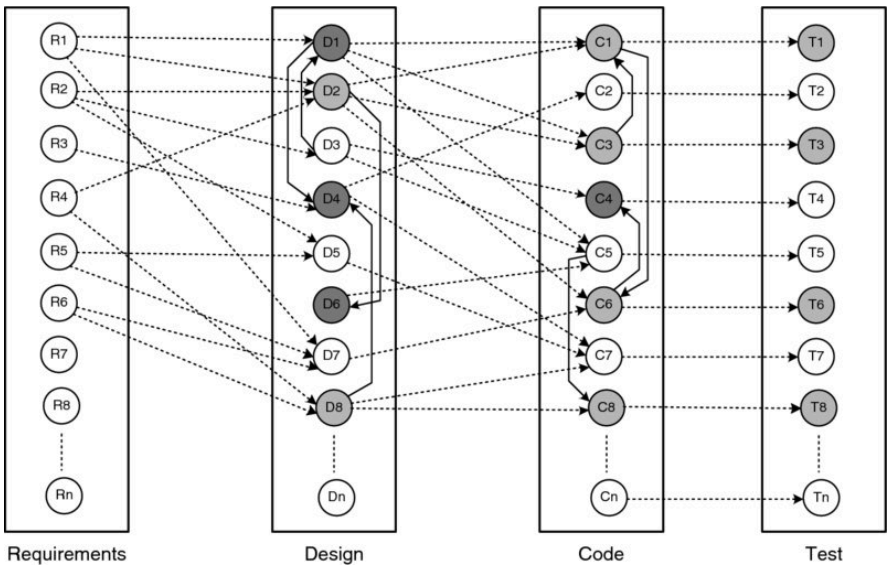
Attribute	Description
Associated Feature	Search feature

Description	Search returns no results when the query contains special characters such as %, &, and @
Maintenance Type	Corrective
Priority	High
Severity	Critical
Time to Implement	1.5 person-weeks
Verification Method	Testing and inspection

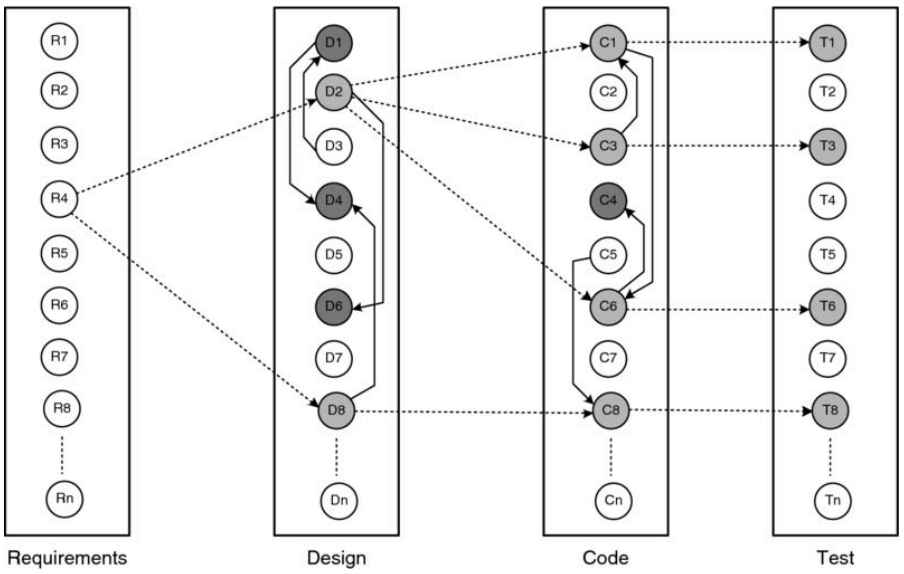
- You must have at least 8 change requests, categorized by type as follows.

Type of change	Number of change requests needed
Corrective	2
Adaptive	2
Perfective	2
Preventive	2

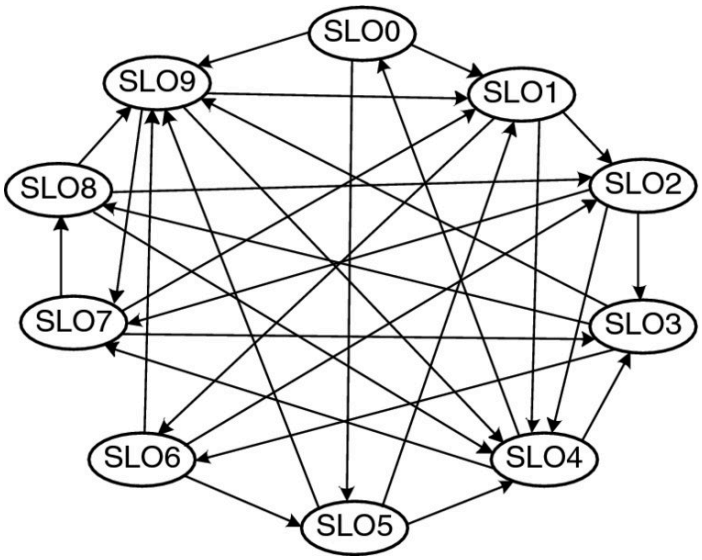
- **D4: Impact Analysis**
  - Create a file called D4_IMPACT_ANALYSIS.md for this deliverable. The file must include the artifacts described below.
  - Based on the given three new features and the analyzed change requests, you must create a traceability graph of your software (similar to what is shown below). The design can be the containers in your C4 diagram. The code can be each package or module of code that you have. The test is a set of test cases for the code. You can use any tool to create this graph.



- Then, you must create another version of the traceability graph that includes only the parts affected by the changes (as shown below).



- Next, only focus on the code modules. We will call each code module an SLO (software lifecycle object). Create a directed graph of SLOs as shown below.



- Then, create a connectivity matrix with distances (as shown below).

	SLO0	SLO1	SLO2	SLO3	SLO4	SLO5	SLO6	SLO7
SLO0		1	2	3	2	1	2	2
SLO1	2		1	2	1	2	1	2
SLO2	2	2		1	1	3	2	1
SLO3	3	2	2		2	2	1	2
SLO4	1	2	3	1		2	2	1
SLO5	2	1	2	2	1		2	2
SLO6	3	2	1	2	2	1		2
SLO7	3	1	2	1	2	3	2	
SLO8	3	2	1	2	1	3	3	2
SLO9	2	1	2	2	1	3	2	1

    - Lastly, include a section in the D4_IMPACT_ANALYSIS.md to list the following:
        - Which change requests are easy to apply and why?
        - Which change requests are difficult to apply and why?
        - To make the maintenance easier, what would you expect from the previous developers?

- **D5: AI-Usage Report**
    - You can use the AI coding agents to help you complete this project. However, you must report on how to use such AIs.
    - Create a file called D5_AI-USAGE.md and list the activities that you use the AI to help you with in this project.

Presentation

The Project Phase 2 Part 1 will be presented on **1 May 2026 during class time (9:00 - 12:00, this is a make-up class**. The presentation must include:

1. A live demo of D1: the working, well-maintained project.
2. An explanation of D2, D3, D4, and D5.

The schedule of the project presentation will be announced later.

Grading Criteria

Category	Weight	Criteria
1. Successful maintenance	20%	The project must include all requested changes and be successfully executed and deployed.
2. The code quality	25%	The SonarQube report must show that the project's quality is not “lower” than the original version.
3. The change request analysis	25%	You show that the change request analysis has been correctly performed.
4. The impact analysis	25%	You appropriately show that the impact analysis has been correctly performed.
5. The given feature request	5%	Your group will be evaluated based on the appropriateness and the feasibility of the feature requests given to your paired group. 5% will be given if the features that you request are appropriate and feasible within the given time frame.

Additional Notes

Points will be deducted in the following cases.

- Only work submitted before the deadline will be graded. If your team has no submission before the deadline, the score is zero.
- All team members must attend the project presentation. Any student who is absent will receive zero for the presentation component.
- If you do not follow the submission and formatting instructions, points will be deducted.
- If your team faces issues that may delay progress, you must report them before each deadline. Late reports may not be accepted.
- Any issues not covered by the rules above will be reviewed by both instructors. The decision and any penalty decided by the instructors are final.
- If plagiarism is found, the plagiarizing group will receive a score of 0 for the assignment.